



Python – Collections

Stephan Karrer

Autor: Stephan Karrer

Strings als Sequenzen

```
word = 'Python'

# Zugriff via Index
word[0]          # character in position 0: P
word[5]          # character in position 5: n

word[-1]         # last character: n
word[-2]         # second-last character: o

# Slicing
word[0:2]        # characters from position 0 (included) to 2 (excluded)
word[2:5]        # characters from position 2 (included) to 5 (excluded)
word[:2]         # character from the beginning to position 2 (excluded)
word[4:]         # characters from position 4 (included) to the end
word[-2:]        # characters from the second-last (included) to the end
```

- Strings sind wie einige andere (Listen, Tupel, ...) Sequenz-Typen und weisen die allen Sequenzen gemeinsame Funktionalität auf

Methodik von Strings

```
wort = 'Python'  
wort[0] = 'B'
```

```
Liefert zur Laufzeit Fehler:  
Traceback (most recent call last):  
  File "E:\stephan\workspaces\pycharm\PythonProject\Zahlen.py",  
    line 74, in <module>  
      wort[0] = 'B'  
      ~~~~^^^  
TypeError: 'str' object does not support item assignment
```

- Strings können aber nicht direkt manipuliert werden (sind immutable)
- Verarbeitung geht nur über die Methodik der str-Klasse
<https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str>

Gemeinsame Sequenz-Operationen

Operation	Result
<code>x in s</code>	<code>True</code> if an item of <code>s</code> is equal to <code>x</code> , else <code>False</code>
<code>x not in s</code>	<code>False</code> if an item of <code>s</code> is equal to <code>x</code> , else <code>True</code>
<code>s + t</code>	the concatenation of <code>s</code> and <code>t</code>
<code>s * n</code> or <code>n * s</code>	equivalent to adding <code>s</code> to itself <code>n</code> times
<code>s[i]</code>	<i>i</i> th item of <code>s</code> , origin 0
<code>s[i:j]</code>	slice of <code>s</code> from <i>i</i> to <i>j</i>
<code>s[i:j:k]</code>	slice of <code>s</code> from <i>i</i> to <i>j</i> with step <i>k</i>
<code>len(s)</code>	length of <code>s</code>
<code>min(s)</code>	smallest item of <code>s</code>
<code>max(s)</code>	largest item of <code>s</code>

Listen (Klasse list)

```
# Deklaration von Listen: via []
squares = [1, 4, 9, 16, 25]
mixed = ["Hallo", 2, 4, 6, 8]

# Deklaration von Listen: via [x for x in iterable]
liste = [x for x in "Hallo"]
print(liste)

# Deklaration von Listen: via Konstruktor list() or list(iterable)
liste = list("Hallo")
print(liste)
```

- Listen benutzen eckige Klammern
- Listen müssen nicht den gleichen Inhaltstyp haben
- Listen sind Sequenzen und haben deshalb wie Strings die gemeinsamen Sequenz-Operationen (Index, Slicing, ...)

Listen sind veränderbar

```
cubes = [1, 8, 27, 65, 125]    # something's wrong here
cubes[3] = 64                  # replace the wrong value
cubes.append(216)              # add the cube of 6

# Zuweisung via Slice
letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
letters[2:5] = ['C', 'D', 'E']    # ['a', 'b', 'C', 'D', 'E', 'f', 'g']
letters[2:5] = []                # ['a', 'b', 'f', 'g']
letters[:] = []                 # []
```

- Listen und andere veränderbare Sequenztypen haben zusätzliche gemeinsame Funktionalitäten

Gemeinsame Operatoren von veränderbaren Sequenztypen

Operation	Result
<code>s[i] = x</code>	item <i>i</i> of <i>s</i> is replaced by <i>x</i>
<code>del s[i]</code>	removes item <i>i</i> of <i>s</i>
<code>s[i:j] = t</code>	slice of <i>s</i> from <i>i</i> to <i>j</i> is replaced by the contents of the iterable <i>t</i>
<code>del s[i:j]</code>	removes the elements of <code>s[i:j]</code> from the list (same as <code>s[i:j] = []</code>)
<code>s[i:j:k] = t</code>	the elements of <code>s[i:j:k]</code> are replaced by those of <i>t</i>
<code>del s[i:j:k]</code>	removes the elements of <code>s[i:j:k]</code> from the list
<code>s += t</code>	extends <i>s</i> with the contents of <i>t</i> (for the most part the same as <code>s[len(s):len(s)] = t</code>)
<code>s *= n</code>	updates <i>s</i> with its contents repeated <i>n</i> times

- Zusätzlich gibt es noch gemeinsame Methodik
<https://docs.python.org/3/library/stdtypes.html#mutable-sequence-types>

Verkürzte Zuweisungen

Bezeichner	str
=	a = b ;
+=	a += b ; // entspricht: a = a + b ;
-=	a -= b ; // entspricht: a = a - b ;
*=	a *= b ; // entspricht: a = a * b ;
/=	a /= b ; // entspricht: a = a / b ;
%=	a %= b ; // entspricht: a = a % b ;
...	

- Sofern möglich, sollten immer die verkürzten Varianten benutzt werden, da der Python-Interpreter in diesem Fall den 2-mal referenzierten Wert nur einmal auswertet (ziemlich schwache Leistung)

Tupel (Klasse tuple)

```
# Deklaration von Tupeln: via ()
squares = (1, 4, 9, 16, 25)
mixed = ("Hallo", 2, 4, 6, 8)

# Deklaration von Tupeln: via Konstruktor tuple() or tuple(iterable)
tupel = list("Hallo")
print(liste)
```

- Tupel benutzen runde Klammern
- Tupel sind nicht veränderbare Sequenzen, die Inhalte selbst sind aber möglicherweise veränderbar
- Tupel müssen nicht den gleichen Inhaltstyp haben
- Tupel sind Sequenzen und haben deshalb wie Strings und Listen die gemeinsamen Sequenz-Operationen (Index, Slicing, ...) bzgl. immutable Sequenzen

Noch ein immutable Sequenz-Typ: Ranges

```
list(range(10))           # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
list(range(1, 11))        # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
list(range(0, 30, 5))      # [0, 5, 10, 15, 20, 25]
list(range(0, 10, 3))      # [0, 3, 6, 9]
list(range(0, -10, -1))    # [0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
list(range(0))             # []
list(range(1, 0))          # []
sum(range(1,10))           # 45
```

- Liefern fortlaufende Ganzzahlen (oder auch dynamische Sequenzen von anderen Typen, sofern die Typklassen entsprechende Methodik vorsehen).
- Die Schrittweite (auch negativ) kann durch den optionalen 3. Parameter vorgegeben werden.
- Brauchen wenig Speicher, da die Werte fortlaufend generiert werden

Ein weiterer Collection-Typ: Set (Menge)

```
# Deklaration von Sets: via {}
squares = {1, 4, 9, 16, 25}
mixed = {"Hallo", 2, 4, 6, 8}
unpacked = {1, 2, *[3, 4]}

# Deklaration von Sets: via Konstruktor set() or set(iterable)
s = set()
s = set([1, 2, 3])
s = set("Hallo")
print(s)          # {'o', 'H', 'l', 'a'}

# Deklaration von Sets via comprehension
s = {c for c in 'abracadabra' if c not in 'abc'}
print(s)          # {'r', 'd'}
```

- Sets benutzen geschweifte Klammern
- Duplikate sind nicht erlaubt und werden eliminiert
- Müssen nicht den gleichen Inhaltstyp haben

Operationen mit Sets

```
squares = {1, 4, 9, 16, 25}
print(len(squares))      # 5
print(4 in squares)      # True
print(4 not in squares)  # False
for i in squares:
    print(i)
```

```
s = {1, [2, 3]}          # Fehler!
```

Traceback (most recent call last):

```
File "E:\workspaces\pycharm\PythonProject\Mengen.py", line 20, in <module>
    s = {1, [2, 3]}
          ^^^^^^^^^^^
```

TypeError: unhashable type: 'list'

- Sets werden intern via Hashing gespeichert, deshalb auch nicht ordnungs-erhaltend
- Hashing setzt unveränderbare Inhaltstypen voraus !
- Da Mengen ungeordnet sind, können keine weiteren Sequenz-Operationen (Indexing, Slicing, ...) verwendet werden

Unveränderbare Mengen

```
s = {1, 2, 3}
s.add(4)
print(s)    # {1, 2, 3, 4}

sfrozen = frozenset(1, 2, 3)
sfrozen.add(4)    # Fehler
```

Traceback (most recent call last):

File "E:\workspaces\pycharm\PythonProject\Mengen.py", line 31, in <module>

s = frozenset(1, 2, 3)

TypeError: frozenset expected at most 1 argument, got 3

- Sets sind veränderbar
- Mittels `frozenset` werden unveränderliche Sets bereit gestellt

Weitere Operatoren für Sets und Frozensets

$s \leq t$	True, wenn es sich bei der Menge s um eine Teilmenge der Menge t handelt, andernfalls False
$s < t$	True, wenn es sich bei der Menge s um eine echte Teilmenge* der Menge t handelt, andernfalls False
$s \geq t$	True, wenn es sich bei der Menge t um eine Teilmenge der Menge s handelt, andernfalls False
$s > t$	True, wenn es sich bei der Menge t um eine echte Teilmenge der Menge s handelt, andernfalls False
$s t$	Erzeugt eine neue Menge, die alle Elemente von s und t enthält. Diese Operation bildet also die Vereinigungsmenge zweier Mengen.
$s \& t$	Erzeugt eine neue Menge, die die Objekte enthält, die sowohl Element der Menge s als auch Element der Menge t sind. Diese Operation bildet also die Schnittmenge zweier Sets.
$s - t$	Erzeugt eine neue Menge mit allen Elementen von s , außer denen, die auch in t enthalten sind. Diese Operation erzeugt also die Differenz zweier Mengen.
$s \wedge t$	Erzeugt eine neue Menge, die alle Objekte enthält, die entweder in s oder in t vorkommen, nicht aber in beiden. Diese Operation bildet also die symmetrische Differenz** zweier Mengen.

Weitere Methodik von Sets und Frozensets

Methode	Beschreibung
<code>s.issubset(t)</code>	äquivalent zu $s \leq t$
<code>s.issuperset(t)</code>	äquivalent zu $s \geq t$
<code>s.isdisjoint(t)</code>	Prüft, ob die Mengen s und t disjunkt sind, das heißt, ob sie eine leere Schnittmenge haben.
<code>s.union(t)</code>	äquivalent zu $s \mid t$
<code>s.intersection(t)</code>	äquivalent zu $s \& t$
<code>s.difference(t)</code>	äquivalent zu $s - t$
<code>s.symmetric_difference(t)</code>	äquivalent zu $s \wedge t$
<code>s.copy()</code>	Erzeugt eine Kopie des Sets s .

- Bei den Methoden der Klasse `set` bzw. `frozenset` kann der Parameter auch eine Sequenz sein !

Auch hier existieren kompakte Zuweisungen

Operator	Entsprechung
<code>s = t</code>	<code>s = s t</code>
<code>s &= t</code>	<code>s = s & t</code>
<code>s -= t</code>	<code>s = s - t</code>
<code>s ^= t</code>	<code>s = s ^ t</code>

- Aus Performanzgründen sollen diese verwendet werden!

Veränderliche Menge: set

```
s = {1, 2, 3}
s.add(4)
print(s)      # {1, 2, 3, 4}

s.remove(4) # wirft exception falls nicht vorhanden
print(s)     # {1, 2, 3}

s.discard(4) # ignoriert nicht vorhanden
print(s)     # {1, 2, 3}

s.clear()
print(s)     # set()
```

- Sets sind veränderbar und können somit nicht selbst Elemente von Sets sein!
- Weitere Methodik siehe <https://docs.python.org/3/library/stdtypes.html#set>

Packen und Entpacken von Sequenzen und Mengen

```
# Packing
datum = 26, 7, 1987
print(datum)      # (26, 7, 1987)

# Unpacking
(tag, monat, jahr) = datum
print(tag)        # 26

# Elemente-Tausch
a, b = 10, 20
a, b = b, a
a, b, c = "xyz"
a, c = b, c
print(a, b, c)    # y y z
```

- Das automatische Packen/Entpacken kann sowohl für Sequenzen als auch Tupel verwendet werden

Packen und Entpacken mit *

```
zahlen = [11, 18, 12, 15, 10]
*erste, x, y = zahlen
print(erste, x, y)          # [11, 18, 12] 15 10

x, y, *rest = zahlen
print(x, y, rest)          # 11 18 [12, 15, 10]

x, *sonstige, y = zahlen
print(x, sonstige, y)      # 11 [18, 12, 15] 10
```

- Mit * können die restlichen Elemente adressiert werden
- Es darf in einer Zuweisung nur eine Variable mit * verwendet werden!

Key-Value: Dictionary

```
# Deklaration von Dictionaries: via {}
squares = {"zahl1":1, "zahl2": 4, "zahl3":9}
mixed = {"el1":"Hallo",      # Einträge zeilenweise
        "zahl2": 4,
        "zahl3":9}
unpacked = {"a": 1, **{"b": 2, "c": 3}}
```



```
# Deklaration von Dictionaries: via Konstruktor dict() or dict(iterable)
d = dict(Germany="Deutschland", Spain="Spanien")
d = dict([("Germany", "Deutschland"), ("Spain", "Spanien")])
```



```
# Deklaration von Dictionaries via comprehension
d = {i: i*i for i in range(5)}
print(d)          # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

- Dictionaries benutzen geschweifte Klammern und Key-Value-Paare

Eindeutigkeit von Keys

```
d = {[1,2,3]: "abc"}          # Fehler!
```

```
Traceback (most recent call last):
```

```
  File "E:\workspaces\pycharm\PythonProject\Maps.py", line 20, in <module>
```

```
    s = {1, [2, 3]}
```

```
          ^^^^^^^^^^^
```

```
TypeError: unhashable type: 'list'
```

- Hashing setzt unveränderbare Inhaltstypen voraus, Key-Typ muss unveränderbar sein!
- Der Werte-Typ kann beliebig sein und es sind auch Duplikate erlaubt
- Dictionaries sind bzgl. der Einfügensreihenfolge seit Version 3.7 ordnungserhaltend

Iteration bei Dictionaries

```
d = dict(Germany="Deutschland", Spain="Spanien")

for key in d:          # Iteration über die Keys
    print(key)

for value in d.values(): # Iteration über die Values
    print(value)

for key, value in d.items(): # Iteration über die Paare
    print(key, "->", value)
```

- Es kann sowohl über die Keys, Values als auch die Key-Value-Paare iteriert werden

Operationen mit Dictionaries

Operator	Beschreibung
<code>len(d)</code>	Liefert die Anzahl aller im Dictionary <code>d</code> enthaltenen Schlüssel-Wert-Paare.
<code>d[k]</code>	Zugriff auf den Wert mit dem Schlüssel <code>k</code>
<code>del d[k]</code>	Löschen des Schlüssels <code>k</code> und seines Wertes
<code>k in d</code>	True, wenn sich der Schlüssel <code>k</code> in <code>d</code> befindet
<code>k not in d</code>	True, wenn sich der Schlüssel <code>k</code> nicht in <code>d</code> befindet
<code>d1 d2</code>	Kombiniert zwei Dictionaries <code>d1</code> und <code>d2</code> . Neu in Python 3.9.

Weitere Methodik von Dictionaries

Methode	Beschreibung
<code>d.clear()</code>	Leert das Dictionary <code>d</code> .
<code>d.copy()</code>	Erzeugt eine Kopie von <code>d</code> .
<code>d.get(k, [x])</code>	Liefert <code>d[k]</code> , wenn der Schlüssel <code>k</code> vorhanden ist, ansonsten <code>x</code> .
<code>d.items()</code>	Gibt ein iterierbares Objekt zurück, das alle Schlüssel-Wert-Paare von <code>d</code> durchläuft.
<code>d.keys()</code>	Gibt ein iterierbares Objekt zurück, das alle Schlüssel von <code>d</code> durchläuft.
<code>d.pop(k)</code>	Gibt den zum Schlüssel <code>k</code> gehörigen Wert zurück und löscht das Schlüssel-Wert-Paar aus dem Dictionary <code>d</code> .
<code>d.popitem()</code>	Gibt ein willkürliches Schlüssel-Wert-Paar von <code>d</code> zurück und entfernt es aus dem Dictionary.
<code>d.setdefault(k, [x])</code>	Das Gegenteil von <code>get</code> . Setzt <code>d[k] = x</code> , wenn der Schlüssel <code>k</code> nicht vorhanden ist.